

```

i = k; //Xuất phát từ vị trí thứ k
while ( i > 0 && ai == n-k+i) //Xác định i để ai ≠ n-k+i
    i = i -1;
if (i>0) { //Nếu chưa phải là tổ hợp cuối cùng thì i>0
    ai = ai + 1;
    for ( j = i+1; j <=k; j++)
        aj = ai + j - i;
    }
else OK =False; ///Nếu là tổ hợp cuối cùng thì i=0

```

Dưới đây là chương trình liệt kê tổ hợp chập k của 1, 2, ..., n.

Chương trình cài đặt thuật toán sinh tập con k phần tử được thể hiện như dưới đây. Trong đó, thuật toán sinh tổ hợp kế tiếp có tên là Next_Combination().

```

#include <iostream>
#include <iomanip>
#define MAX 100
int X[MAX], n, k, dem=0;
bool OK = true;
using namespace std;
void Init(void){ //thiết lập tập con đầu tiên
    cout<<"\n Nhập n, k:"; cin>>n>>k;
    for(int i=1; i<=k; i++) //tập con đầu tiên là 1, 2, ..., k
        X[i] = i;
}

void Result(void){ //đưa ra tập con hiện tại
    cout<<"\n Kết quả "<<dem<<":";
    for(int i=1; i<=k; i++) //đưa ra X[] =( x1, x2, ..., xk)
        cout<<X[i]<<setw(3);
    }

void Next_Combination(void){ //sinh tập con k phần tử từ tập con bất kỳ
    int i = k; //duyet từ vị trí bên phải nhất của tập con
    while(i>0 && X[i]== n-k+i) //tìm i sao cho xi ≠ n-k+i
        i--;
}

```

```

    if (i>0){//nếu chưa phải là tập con cuối cùng
        X[i]= X[i]+1; //thay đổi giá trị tại vị trí i:  $x_i = x_i + 1$ ;
        for(int j=i+1; j<=k; j++) //các vị trí j từ i+1,..., k
            X[j] = X[i] + j - i; // được thay đổi là  $x_j = x_i + j - i$ ;
        }
    else //nếu là tập con cuối cùng
        OK = false; //ta kết thúc duyệt
    }
}
int main(void){
    Init(); //khởi tạo cấu hình đầu tiên
    while(OK){ //lặp trong khi cấu hình chưa phải cuối cùng
        Result(); //đưa ra cấu hình hiện tại
        Next_Combination(); //sinh ra cấu hình kế tiếp
    }
}

```

Ví dụ 3. Liệt kê các hoán vị của tập n phần tử. Cho $X = \{ 1, 2, \dots, n \}$. Hãy liệt kê các hoán vị từ n phần tử của X .

Lời giải : Mỗi hoán vị từ n phần tử của X có thể biểu diễn bởi bộ có thứ tự n thành phần

$a = (a_1, a_2, \dots, a_n)$ thoả mãn $a_i \in X, i = 1, 2, \dots, n, a_p \neq a_q, p \neq q$.

Trên tập các hoán vị từ n phần tử của X có thể xác định nhiều thứ tự khác nhau. Tuy nhiên, thứ tự dễ thấy nhất là thứ tự từ điển được định nghĩa như sau:

Ta nói hoán vị $a = a_1 a_2 \dots a_n$ đi trước hoán vị $a' = a'_1 a'_2 \dots a'_n$ trong thứ tự từ điển và ký hiệu là $a < a'$, nếu tìm được chỉ số k ($1 \leq k \leq n$) sao cho

$a_1 = a'_1, a_2 = a'_2, \dots, a_{k-1} = a'_{k-1}, a_k < a'_k$.

Chẳng hạn $X = \{ 1, 2, 3, 4 \}$. Các hoán vị các phần tử của X được liệt kê theo thứ tự từ điển như sau:

1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

Như vậy, hoán vị đầu tiên trong thứ tự từ điển là $(1, 2, \dots, n)$ và hoán vị cuối cùng là $(n, n-1, \dots, 1)$. Giả sử $a = a_1 a_2 \dots a_n$ là một hoán vị chưa phải là cuối cùng. Khi đó ta

có thể chứng minh được rằng, hoán vị kế tiếp trong thứ tự từ điển có thể xây dựng bằng cách thực hiện các qui tắc biến đổi sau đối với hoán vị hiện tại:

- ☐ Tìm từ phải qua trái hoán vị có chỉ số j đầu tiên thoả mãn $a_j < a_{j+1}$ (hay j là chỉ số lớn nhất để $a_j < a_{j+1}$);
- ☐ Tìm a_k là số nhỏ nhất còn lớn hơn a_j trong các số ở bên phải a_j ;
- ☐ Đổi chỗ a_j với a_k
- ☐ Lật ngược đoạn từ a_{j+1} đến a_n .

Chẳng hạn ta đang có hoán vị (3, 6, 2, 5, 4, 1), cần xây dựng hoán vị kế tiếp theo thứ tự từ điển. Ta duyệt từ $j = n-1$ sang bên trái để tìm j đầu tiên thoả mãn $a_j < a_{j+1}$ ta nhận được $j=3$ ($a_3=2 < a_4=5$). Số nhỏ nhất còn lớn hơn a_3 trong các số bên phải a_3 là a_5 ($a_5=4$). Đổi chỗ a_3 cho a_5 ta thu được (3, 6, 4, 5, 2, 1), lật ngược đoạn từ a_4 đến a_6 ta nhận được (3,6,4,1,2,5).

Từ đó thuật toán sinh kế tiếp có thể được mô tả bằng thủ tục sau:

Thuật toán sinh hoán vị kế tiếp:

```
void Next_Permutation( void ){
    j = n-1; //Duyệt từ vị trí j=n-1
    while (j > 0 && a_j > a_{j+1}) //Tìm vị trí j để a_j > a_{j+1}
        j = j - 1;
    if (j > 0) { // Nếu j > 0 thì hoán vị chưa phải cuối cùng
        k = n; //Xuất phát từ vị trí k=n
        while (a_j > a_k) //Tìm k để a_j < a_k
            k = k - 1;
        temp = a_j; a_j = a_k; a_k = temp; //Đổi chỗ a_j cho a_k
        r = j + 1; s = n;
        while ( r < s) { //Lật ngược lại đoạn từ j+1 đến n
            temp = a_r; a_r = a_s; a_s = temp;
            r = r + 1; s = s - 1;
        }
    }
    else OK = False; //Nếu là hoán vị cuối cùng thì i=0
}
```

Chương trình liệt kê hoán vị được thể hiện như sau:

```
#include <iostream>
```

```

#include <iomanip>
#define MAX 100
int X[MAX], n, dem=0;
bool OK = true;
using namespace std;
void Init(void){ //thiết lập hoán vị đầu tiên
    cout<<"\n Nhập n:"; cin>>n;
    for(int i=1; i<=n; i++) //thiết lập X[] = (1, 2, ...,n)
        X[i] = i;
}
void Result(void){ //đưa ra hoán vị hiện tại
    cout<<"\n Kết quả "<<dem<<":";
    for(int i=1; i<=n; i++)
        cout<<X[i]<<setw(3);
}
void Next_Permutation(void){ //sinh ra hoán vị kế tiếp
    int j = n-1; //xuất phát từ vị trí j = n-1
    while(j>0 && X[j]>X[j+1]) //tìm chỉ số j sao cho X[j] < X[j+1]
        j--;
    if (j > 0){ // nếu chưa phải hoán vị cuối cùng
        int k = n; //xuất phát từ vị trí k = n
        while(X[j]>X[k]) //tìm chỉ số k sao cho X[j] < X[k]
            k--;
        int t = X[j]; X[j] = X[k]; X[k]=t; //đổi chỗ X[j] cho X[k]
        int r = j+1, s = n;
        while (r<=s){ //lật ngược lại đoạn từ j+1,...,n
            t=X[r]; X[r]=X[s]; X[s]=t;
            r++; s--;
        }
    }
    else //nếu là cấu hình cuối cùng
        OK = false; //ta kết thúc duyệt
}
int main(void){ //đây là thuật toán sinh
    Init(); //thiết lập cấu hình đầu tiên
    while(OK){ //lặp trong khi cấu hình chưa phải cuối cùng
        Result(); //đưa ra cấu hình hiện tại

```

```

        Next_Permutation(); //sinh ra cấu hình kế tiếp
    }
}

```

Ví dụ 4. Bài toán: Cho n là số nguyên dương. Một cách phân chia số n là biểu diễn n thành tổng các số tự nhiên không lớn hơn n . Chẳng hạn $8 = 2 + 3 + 2$.

Lời giải. Hai cách chia được gọi là đồng nhất nếu chúng có cùng các số hạng và chỉ khác nhau về thứ tự sắp xếp. Chọn cách phân chia số $n = b_1 + b_2 + \dots + b_k$ với $b_1 > b_2 > \dots > b_k$, và duyệt theo trình tự từ điển ngược. Chẳng hạn với $n = 5$, chúng ta có thứ tự từ điển ngược của các cách phân chia như sau:

5				
4	1			
3	2			
3	1	1		
2	2	1		
2	1	1	1	1
1	1	1	1	1

Như vậy, cách chia đầu tiên chính là n . Cách chia cuối cùng là dãy n số 1. Bây giờ chúng ta chỉ cần xây dựng thuật toán sinh kế tiếp cho mỗi cách phân chia chưa phải là cuối cùng.

Thuật toán sinh cách phân chia kế tiếp:

```

void Next_Division(void){
    int i, j, R, S, D;
    i = k; //Xuất phát từ cuối cách chia trước đó
    while(i>0 && C[i]==1) //Tìm i sao cho C[i]≠1
        i--;
    if(i>0){ // Nếu chưa phải là cách chia cuối cùng thì i>0
        C[i] = C[i]-1; //Giảm C[i] đi một đơn vị.
        D = k - i + 1;
        R = D / C[i];
        S = D % C[i];
        k = i;
        if(R>0){
            for(j=i+1; j<=i+R; j++)
                C[j] = C[i];
            k = k+R;
        }
        if(S>0){

```

```

        k=k+1; C[k] = S;
    }
}
else Stop=TRUE;
}

```

Chương trình liệt kê các cách chia số n thành tổng các số nhỏ hơn:

```

#include <iostream.h>
#include <stdlib.h>
#define MAX 100
#define TRUE 1
#define FALSE 0
int n, k, X[MAX], dem =0, OK =TRUE;
void Init(void ){
    cout<<"\n Nhap n=";>n; k = 1; X[k] = n;
}
void Result(void) {
    cout<<"\n Cach chia "<<++dem<<":";
    for (int i=1; i<=k; i++)
        cout<<X[i]<<" ";
}
void Next_Division(void ){
    int i = k, j, R, S,D;
    while (i > 0 && X[i]==1 ) i--;
    if (i>0 ) {
        X[i] = X[i] - 1;
        D = k - i + 1;
        R = D / X[i];
        S = D % X[i];
        k= i;
        if (R>0) {
            for (j = i +1; j<=i + R; j++) X[j] = X[i];
            k = k + R;
        }
        if (S>0) { k = k +1; X[k] = S; }
    }
    else OK =0;
}
int main() {
    Init();
    while (OK ) {
        Result();
        Next_Division();
    }
}

```

```

    }
    system("PAUSE");
    return 0;
}

```

3.4. Thuật toán quay lui (Back track)

Phương pháp sinh kế tiếp có thể giải quyết được các bài toán liệt kê khi ta nhận biết được cấu hình đầu tiên & cấu hình cuối cùng của bài toán. Tuy nhiên, không phải cấu hình sinh kế tiếp nào cũng được sinh một cách đơn giản từ cấu hình hiện tại, ngay kể cả việc phát hiện cấu hình ban đầu cũng không phải dễ tìm vì nhiều khi chúng ta phải chứng minh sự tồn tại của cấu hình. Do vậy, thuật toán sinh kế tiếp chỉ giải quyết được những bài toán liệt kê đơn giản. Để giải quyết những bài toán tổ hợp phức tạp, người ta thường dùng thuật toán quay lui (Back Track) sẽ được trình bày dưới đây.

Nội dung chính của thuật toán này là xây dựng dần các thành phần của cấu hình bằng cách thử tất cả các khả năng. Giả sử cần phải tìm một cấu hình của bài toán $x = (x_1, x_2, \dots, x_n)$ mà $i-1$ thành phần $x_1, x_2, \dots, x_{i-1}$ đã được xác định, bây giờ ta xác định thành phần thứ i của cấu hình bằng cách duyệt tất cả các khả năng có thể có và đánh số các khả năng từ 1 đến n_i . Với mỗi khả năng j , kiểm tra xem j có chấp nhận được hay không. Khi đó có thể xảy ra hai trường hợp:

- ☐ Nếu chấp nhận j thì xác định x_i theo j , nếu $i=n$ thì ta được một cấu hình cần tìm, ngược lại xác định tiếp thành phần x_{i+1} .
- ☐ Nếu thử tất cả các khả năng mà không có khả năng nào được chấp nhận thì quay lại bước trước đó để xác định lại x_{i-1} .

Điểm quan trọng nhất của thuật toán là phải ghi nhớ lại mỗi bước đã đi qua, những khả năng nào đã được thử để tránh sự trùng lặp. Để nhớ lại những bước duyệt trước đó, chương trình cần phải được tổ chức theo cơ chế ngăn xếp (Last in first out). Vì vậy, thuật toán quay lui rất phù hợp với những phép gọi đệ qui. Thuật toán quay lui xác định thành phần thứ i có thể được mô tả bằng thủ tục Try(i) như sau:

```

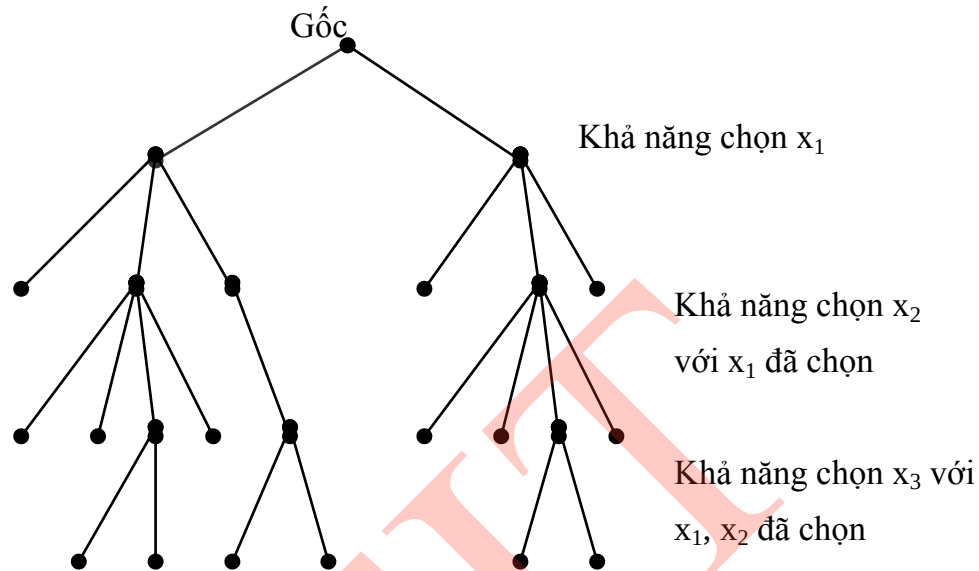
void Try( int i ) {
    for ( j = 1; j < ni; j ++ ) {
        if ( <Chấp nhận j > ) {
            <Xác định xi theo j>
            if ( i == n )
                <Ghi nhận cấu hình>;
            else    Try(i+1);
        }
    }
}

```

}

}

Có thể mô tả quá trình tìm kiếm lời giải theo thuật toán quay lui bằng cây tìm kiếm lời giải sau:

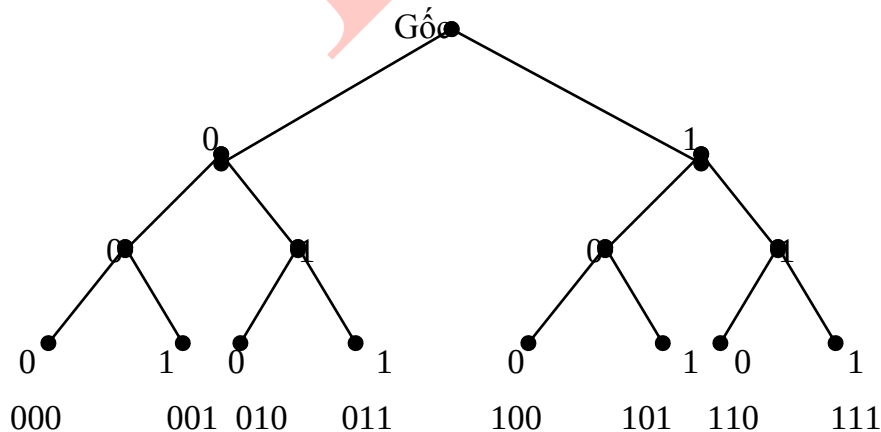


Hình 3.1. Cây liệt kê lời giải theo thuật toán quay lui.

Dưới đây là một số ví dụ điển hình sử dụng thuật toán quay lui.

Ví dụ 1. Liệt kê các xâu nhị phân độ dài n.

Biểu diễn các xâu nhị phân dưới dạng $b_1, b_2, \dots, b_n$, trong đó $b_i \in \{0, 1\}$. Thủ tục đệ qui Try(i) xác định b_i với các giá trị đề cử cho b_i là 0 và 1. Các giá trị này mặc nhiên được chấp nhận mà không cần phải thỏa mãn điều kiện gì (do đó bài toán không cần đến biến trạng thái). Thủ tục Init khởi tạo giá trị n và biến đếm count. Thủ tục kết quả in ra dãy nhị phân tìm được. Chẳng hạn với $n=3$, cây tìm kiếm lời giải được thể hiện như hình 3.2.



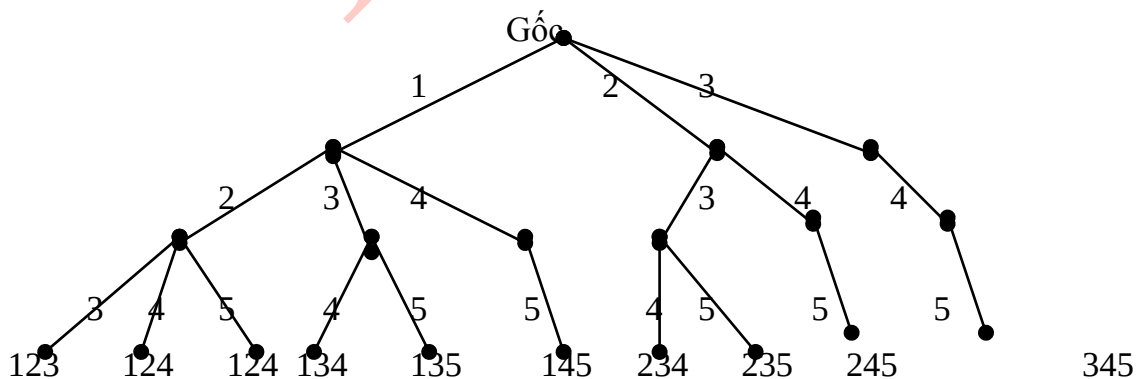
Hình 3.2. Cây tìm kiếm lời giải liệt kê dãy nhị phân độ dài 3

Văn bản chương trình liệt kê các xâu nhị phân có độ dài n sử dụng thuật toán quay lui được thực hiện như sau:

```
#include <iostream >
#define MAX 100
#define TRUE 1
#define FALSE 0
int n, X[MAX], dem=0;
void Init (void){    cout<<"\n Nhap n=";<cin>>n;}
void Result(void){
    cout<<"\n Ket qua buoc "<<++dem<<":";
    for (int i=1; i<=n; i++)
        cout<<X[i]<<" ";
}
void Try (int i) {
    for (int j=0; j<=1; j++){
        X[i] = j;
        if (i==n) Result();
        else Try(i+1);
    }
}
int main(){
    Init(); //Nhap n = 4
    Try(1);system("PAUSE");return 0;
}
```

Ví dụ 2. Liệt kê các tập con k phần tử của tập n phần tử

Giải. Biểu diễn tập con k phần tử dưới dạng $c_1, c_2, \dots, c_k$, trong đó $1 < c_1 < c_2 < \dots < c_k \leq n$. Từ đó suy ra các giá trị đề cử cho c_i là từ $c_{i-1} + 1$ cho đến $n - k + i$. Cần thêm vào $c_0 = 0$. Các giá trị đề cử này mặc nhiên được chấp nhận mà không cần phải thêm điều kiện gì. Các thủ tục Init, Result được xây dựng như những ví dụ trên. Cây tìm kiếm lời giải bài toán liệt kê tập con k phần tử của tập n phần tử với $n=5, k=3$ được thể hiện như trong hình 3.3.



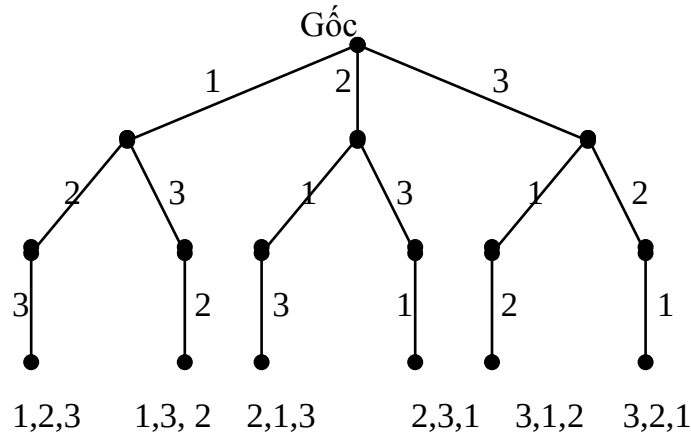
Hình 3.3. Cây liệt kê tổ hợp chập 3 từ $\{1, 2, 3, 4, 5\}$

Chương trình liệt kê các tập con k phần tử trong tập n phần tử được thể hiện như sau:

```
#include <iostream.h>
#include <stdlib.h>
#define MAX 100
#define TRUE 1
#define FALSE 0
int n,k,X[MAX],dem=0;
void Init (void ){
    cout<<"\n Nhap n=";>>n;
    cout<<"\n Nhap k=";>>k;
    X[0] = 0;
}
void Result(void){
    cout<<"\n Ket qua buoc "<<++dem<<": ";
    for (int i=1; i<=k; i++)
        cout<<X[i]<<" ";
}
void Try(int i ) {
    for (int j = X[i-1]+1; j<= n-k+i; j++){
        X[i] = j;
        if (i==k) Result();
        else Try(i+1);
    }
}
int main(){
    Init(); //Nhap n = 5, k = 3
    Try(1);
    system("PAUSE");
    return 0;
}
```

Ví dụ 3. Liệt kê các hoán vị của tập n phần tử.

Lời giải. Biểu diễn hoán vị dưới dạng $p_1, p_2, \dots, p_n$, trong đó p_i nhận giá trị từ 1 đến n và $p_i \neq p_j$ với $i \neq j$. Các giá trị từ 1 đến n lần lượt được đề cử cho p_i , trong đó giá trị j được chấp nhận nếu nó chưa được dùng. Vì vậy, cần phải ghi nhớ với mỗi giá trị j xem nó đã được dùng hay chưa. Điều này được thực hiện nhờ một dãy các biến logic b_j , trong đó $b_j = \text{true}$ nếu j chưa được dùng. Các biến này phải được khởi đầu giá trị true trong thủ tục Init. Sau khi gán j cho p_i , cần ghi nhận false cho b_j và phải gán true khi thực hiện xong Result hay Try(i+1). Các thủ tục còn lại giống như ví dụ 1, 2. Hình 3.4 mô tả cây tìm kiếm lời giải bài toán liệt kê hoán vị của 1, 2, ..., n với n = 3.



Hình 3.4. Cây tìm kiếm lời giải bài toán liệt kê hoán vị của $\{1,2,3\}$

Sau đây là chương trình giải quyết bài toán liệt kê các hoán vị của 1, 2, ..., n.

```

#include <iostream.h>
#include <stdlib.h>
#define MAX 100
#define TRUE 1
#define FALSE 0
int n,X[MAX],chuaxet[MAX],dem=0;
void Init (void ){
    cout<<"\n Nhap n=";>n;
    for (int i=1; i<=n; i++) chuaxet[i] = TRUE;
}
void Result(void){
    cout<<"\n Ket qua buoc "<<dem<<":";
    for (int i=1; i<=n; i++) cout<<X[i]<<" ";
}
void Try(int i){
    for (int j =1; j<=n; j++){
        if (chuaxet[j]) {
            X[i] = j; chuaxet[j] = FALSE;
            if (i == n) Result();
            else Try(i+1);
            chuaxet[j] = TRUE;
        }
    }
}
}
int main(){ Init(); //Nhap n = 4
    Try (1); system("PAUSE"); return 0;
}
  
```

Ví dụ 4. Bài toán Xếp Hậu. Liệt kê tất cả các cách xếp n quân hậu trên bàn cờ $n \times n$ sao cho chúng không ăn được nhau.

Lời giải. Bàn cờ có n hàng được đánh số từ 1 đến n , n cột được đánh số từ 1 đến n ; Bàn cờ có $n^2 - 1$ đường chéo xuôi được đánh số từ 1 đến $2*n - 1$, $2 * n - 1$ đường chéo ngược được đánh số từ 1 đến $2*n - 1$. Ví dụ: với bàn cờ 8×8 , chúng ta có 8 hàng được đánh số từ 1 đến 8, 8 cột được đánh số từ 1 đến 8, 15 đường chéo xuôi, 15 đường chéo ngược được đánh số từ 1 . . 15.

Vì trên mỗi hàng chỉ xếp được đúng một quân hậu, nên chúng ta chỉ cần quan tâm đến quân hậu được xếp ở cột nào. Từ đó dẫn đến việc xác định bộ n thành phần $x_1, x_2, \dots, x_n$, trong đó $x_i = j$ được hiểu là quân hậu tại dòng i xếp vào cột thứ j . Giá trị của i được nhận từ 1 đến n ; giá trị của j cũng được nhận từ 1 đến n , nhưng thỏa mãn điều kiện ô (i, j) chưa bị quân hậu khác chiếu đến theo cột, đường chéo xuôi, đường chéo ngược.

Việc kiểm soát theo hàng ngang là không cần thiết vì trên mỗi hàng chỉ xếp đúng một quân hậu. Việc kiểm soát theo cột được ghi nhận nhờ dãy biến logic $chuaxet_j$ với qui ước $chuaxet_j = \text{True}$ nếu cột j còn trống, $chuaxet_j = \text{False}$ nếu cột j không còn trống. Để ghi nhận đường chéo xuôi và đường chéo ngược có chiếu tới ô (i, j) hay không, ta sử dụng phương trình $i + j = \text{const}$ và $i - j = \text{const}$, đường chéo thứ nhất được ghi nhận bởi dãy biến $XUOI_j$, đường chéo thứ 2 được ghi nhận bởi dãy biến $NGUOC_j$ với qui ước nếu đường chéo nào còn trống thì giá trị tương ứng của nó là True ngược lại là False. Như vậy, cột j được chấp nhận khi cả 3 biến $chuaxet_j, XUOI_{i+j}, NGUOC_{i-j}$ đều có giá trị 1. Các biến này phải được khởi đầu giá trị True trước đó, gán lại giá trị False khi xếp xong quân hậu thứ i và trả lại giá trị 1 khi đưa ra kết quả.

Dưới đây là chương trình bằng thuật toán quay lui.

```
#include <iostream.h>
#include <stdlib.h>
#define MAX 100
#define TRUE 1
#define FALSE 0
int X[MAX], XUOI[MAX], NGUOC[MAX], chuaxet[MAX];
int n, dem = 0;
void Init (void) {
    cout<<"\n Nhap n = "; cin>>n;
    for (int i=1; i<=n; i++) chuaxet[i]=TRUE;
    for (int i=1; i<=(2*n-1); i++) {
        XUOI[i] = TRUE; NGUOC[i]=TRUE;
    }
}
void Result(void) {
    cout<<"\n Phuon an "<< ++dem<<": ";
    for (int i=1; i<=n; i++) cout<<X[i]<<" ";
}
```

```

void Try(int i){
    for (int j =1; j<=n; j++){
        if (chuaxet[j] && XUOI[i-j+n] && NGUOC[i+j-1]){
            X[i] = j; chuaxet[j]=FALSE;
            XUOI[i-j+n]=FALSE; NGUOC[i+j-1]=FALSE;
            if (i==n ) Result();
            else Try(i+1);
            chuaxet[j]=TRUE; XUOI[i-j+n]=TRUE;
            NGUOC[i+j-1]=TRUE;
        }
    }
}

int main(){
    Init(); Try(1);
    system("PAUSE");
    return 0;
}

```

Dưới đây là số cách xếp hậu ứng với n.

n	4	7	8	9	10	11	12	13	14
H_n	2	40	92	352	724	2680	14200	73712	365596

Nghiem đầu tiên mà chương trình tìm được ứng với $n=8$ là $x=(1, 5, 8, 6, 3, 7, 2, 4)$ nó tương đương với cách xếp trên hình 5.

3.5. Những nội dung cần ghi nhớ

- ✓ Thế nào là bài toán liệt kê?
- ✓ Những điều kiện bắt buộc của một thuật toán liệt kê.
- ✓ Hiểu và nắm vững lớp các bài toán có thể giải được bằng phương pháp sinh.
- ✓ Hiểu và nắm vững những yếu tố cần thiết để thực hiện giải thuật quay lui.

BÀI TẬP CHƯƠNG 3

1. Liệt kê tất cả các xâu nhị phân độ dài 5 không chứa hai số 0 liên tiếp.
2. Liệt kê tất cả các phần tử của tập

$$D = \{x = (x_1, x_2, \dots, x_n) : \sum_{j=1}^n a_j x_j = b, \quad x_j \in \mathbb{Z}_+, j = 1, 2, \dots, n\}$$

Trong đó $a_1, a_2, \dots, a_n, b$ là các số nguyên dương.

3. **Hình vuông thần bí ma phương bậc n** là ma trận vuông cấp n với các phần tử là các số tự nhiên từ 1 đến n^2 thỏa mãn các tính chất: Tổng các phần tử trên mỗi dòng, mỗi cột và mỗi một trong hai đường chéo có cùng một giá trị. Hãy liệt kê tất cả các ma phương bậc 3, 4 không sai khác nhau bởi các phép biến hình đơn giản (quay, đối xứng).

Ví dụ dưới đây là một ma phương bậc 3 thỏa mãn tính chất tổng hàng, cột, đường chéo đều là 15.

$$\begin{vmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{vmatrix}$$

4. Tam giác thần bí. Cho một lưới ô vuông gồm $n \times n$ ô và số nguyên dương k . Tìm cách điền các số tự nhiên từ 1 đến $3n-3$ vào các ô ở cột đầu tiên, dòng cuối cùng và đường chéo chính sao cho tổng các số điền trong cột đầu tiên, dòng cuối cùng và đường chéo chính của lưới đều bằng k . Ví dụ $n=5, k=35$ ta có cách điền sau:

11				
10	3			
9		2		
1			7	
4	5	6	8	12

Phát triển thuật toán dựa trên thuật toán quay lui để chỉ ra với giá trị của n, k cho trước bài toán có lời giải hay không. Nếu có câu trả lời chỉ cần đưa ra một lời giải.

5. **Tìm tập con dài nhất có thứ tự tăng dần, giảm dần.** Cho dãy số $a_1, a_2, \dots, a_n$. Hãy tìm dãy con dài nhất được sắp xếp theo thứ tự tăng hoặc giảm dần. Dữ liệu vào cho bởi file tapcon.in, dòng đầu tiên ghi lại số tự nhiên n ($n \leq 100$), dòng kế tiếp ghi lại n số, mỗi số được phân biệt với nhau bởi một hoặc vài ký tự rỗng. Kết quả ghi lại trong file tapcon.out. Ví dụ sau sẽ minh họa cho file tapcon.in và tapcon.out.

tapcon.in		tapcon.out
5		5
7 1 3 8 9 6 12		1 3 8 9 12

6. **Duyệt các tập con thỏa mãn điều kiện.** Cho dãy số $a_1, a_2, \dots, a_n$ và số M . Hãy tìm tất cả các dãy con trong dãy số $a_1, a_2, \dots, a_n$ sao cho tổng các phần tử trong dãy con đúng bằng M . Dữ liệu vào cho bởi file tapcon.in, dòng đầu tiên ghi lại hai số tự nhiên N và số M ($N \leq 100$), dòng kế tiếp ghi lại N số mỗi số được phân biệt với nhau bởi một và dấu trống. Kết quả ghi lại trong file tapcon.out. Ví dụ sau sẽ minh họa cho file tapcon.in và tapcon.out

tapcon.in	
7 50	
5 10 15 20 25 30 35	
tapcon.out	
20 30	
15 35	
10 15 25	
5 20 25	
5 15 30	
5 10 35	
5 10 15 20	

7. **Cho lưới hình chữ nhật** gồm $(n \times m)$ hình vuông đơn vị. Hãy liệt kê tất cả các đường đi từ điểm có tọa độ $(0, 0)$ đến điểm có tọa độ (n, m) . Biết rằng, điểm $(0, 0)$ được coi là đỉnh dưới của hình vuông dưới nhất góc bên trái, mỗi bước đi chỉ được phép thực hiện hoặc lên trên hoặc xuống dưới theo cạnh của hình vuông đơn vị. Dữ liệu vào cho bởi file bai14.inp, kết quả ghi lại trong file bai14.out. Ví dụ sau sẽ minh họa cho file bai14.in và bai14.out.

bai14.in	
2 2	
bai14.out	
0 0 1 1	
0 1 0 1	
0 1 1 0	
1 0 0 1	
1 0 1 0	
1 1 0 0	

8. Tìm bộ giá trị rời rạc để hàm mục tiêu $\sin(x_1 + x_2 + \dots + x_k)$ đạt giá trị lớn nhất. Dữ liệu vào cho bởi file bai4.inp, kết quả ghi lại trong file bai4.out.
9. Duyệt mọi phép toán trong tính toán giá trị biểu thức. Viết chương trình nhập từ bàn phím hai số nguyên M, N. Hãy tìm cách thay các dấu ? trong biểu thức sau bởi các phép toán +, -, *, %, / (chia nguyên) sao cho giá trị của biểu thức nhận được bằng đúng N:

((((M?M) ?M)?M)?M)?M

Nếu không được hãy đưa ra thông báo là không thể được.

PDF

CHƯƠNG 4. BÀI TOÁN TỐI ƯU

Bài toán đếm thực hiện đếm các cấu hình tổ hợp thỏa mãn một số tính chất nào đó. Bài toán liệt kê xem xét từng cấu hình tổ hợp thỏa mãn các tính chất đặt ra. Bài toán tối ưu chỉ quan tâm đến nghiệm tốt nhất (xấu nhất) theo một nghĩa nào đó đặt ra của bài toán. Nội dung chính của chương này là giới thiệu các phương pháp giải quyết bài toán tối ưu đồng thời giải quyết một số bài toán có vai trò quan trọng của lý thuyết tổ hợp. Những nội dung được đề cập bao gồm:

- ✓ Giới thiệu bài toán và phát biểu bài toán tối ưu cho các mô hình thực tế.
- ✓ Phương pháp liệt kê giải quyết bài toán tối ưu.
- ✓ Phương pháp nhánh cận giải quyết bài toán tối ưu.
- ✓ Phương pháp quy hoạch động giải quyết bài toán tối ưu.

Bạn đọc có thể tìm thấy phương pháp giải chi tiết cho nhiều bài toán tối ưu quan trọng trong các tài liệu [1], [2].

4.1. Giới thiệu bài toán

Trong nhiều bài toán thực tế, các cấu hình tổ hợp còn được gán một giá trị bằng số đánh giá giá trị sử dụng của cấu hình đối với một mục đích sử dụng cụ thể nào đó. Khi đó xuất hiện bài toán: Hãy lựa chọn trong số tất cả các cấu hình tổ hợp chấp nhận được cấu hình có giá trị sử dụng tốt nhất. Các bài toán như vậy được gọi là bài toán tối ưu tổ hợp. Chúng ta có thể phát biểu bài toán tối ưu tổ hợp dưới dạng tổng quát như sau:

Tìm cực tiểu (hay cực đại) của phiếm hàm $f(x) = \min(\max)$ với điều kiện $x \in D$, trong đó D là tập hữu hạn các phần tử.

- Tập D gọi là tập các phương án của bài toán.
- Mỗi phần tử $x \in D$ được gọi là một phương án.
- Hàm $f(x)$ được gọi là hàm mục tiêu của bài toán.
- Phương án $x^* \in D$ đem lại giá trị nhỏ nhất (lớn nhất) cho hàm mục tiêu được gọi là phương án tối ưu.
- Giá trị $f^* = f(x^*)$ được gọi là giá trị tối ưu của bài toán.

Dưới đây là một số bài toán tối ưu tổ hợp kinh điển. Các bài toán này là những mô hình có nhiều ứng dụng thực tế và giữ vai trò quan trọng trong việc nghiên cứu và phát triển lý thuyết tối ưu hoá tổ hợp.

Bài toán cái túi. Một nhà thám hiểm cần đem theo một cái túi có trọng lượng không quá b . Có n đồ vật có thể đem theo. Đồ vật thứ j có trọng lượng a_j và giá trị sử dụng c_j ($j = 1, \dots, n$).

$2, \dots, n$). Hỏi nhà thám hiểm cần đem theo những đồ vật nào để cho tổng giá trị sử dụng là lớn nhất?

Gọi $A = (a_1, a_2, \dots, a_n)$, $C = (c_1, c_2, \dots, c_n)$ tương ứng với vector trọng lượng và giá trị sử dụng các đồ vật. Khi đó ta có thể xác định được tập các phương án và hàm mục tiêu của bài toán như sau:

Tập các phương án của bài toán: Một phương án của nhà thám hiểm có thể biểu diễn như một vector nhị phân độ dài n : $X = (x_1, x_2, \dots, x_n)$, trong đó $x_i = 1$ có nghĩa là đồ vật thứ i được đem theo, $x_i = 0$ có nghĩa đồ vật thứ i không được đem theo. Tập các cấu nhị phân $X = (x_1, \dots, x_n)$ còn phải thỏa mãn điều kiện tổng trọng lượng không vượt quá b . Nói cách khác, tập phương án D của bài toán được xác định như công thức dưới đây.

$$D = \left\{ X = (x_1, x_2, \dots, x_n) : \sum_{i=1}^n a_i x_i \leq b \right\}$$

Hàm mục tiêu của bài toán: Với mỗi phương án $X \in D$, giá trị sử dụng các đồ vật đem theo là $f(X) = \sum_{i=1}^n c_i x_i$, tổng trọng lượng đồ vật đem theo là $g(X) = \sum_{i=1}^n a_i x_i$. Như vậy bài toán cái túi được phát biểu dưới dạng bài toán tối ưu tổ hợp sau:

Trong số các vector nhị phân độ dài n thỏa mãn điều kiện $g(X) \leq b$, hãy tìm vector X^* để hàm mục tiêu $f(X)$ đạt giá trị lớn nhất. Nói cách khác

$$\text{tìm min} \{ f(X) : X \in D \text{ và } g(x) \leq b \}$$

Bài toán Người du lịch. Một người du lịch muốn đi thăm quan n thành phố $T_1, T_2, \dots, T_n$. Xuất phát từ một thành phố nào đó, người du lịch muốn đi qua tất cả các thành phố còn lại, mỗi thành phố đi qua đúng một lần, rồi quay trở lại thành phố xuất phát. Biết c_{ij} là chi phí đi từ thành phố T_i đến thành phố T_j ($i, j = 1, 2, \dots, n$), hãy tìm hành trình với tổng chi phí là nhỏ nhất cho người du lịch (một hành trình là một cách đi thỏa mãn điều kiện).

Không hạn chế tính tổng quát của bài toán ta giả sử người du lịch luôn xuất phát tại một thành phố cố định (giả sử là thành phố số 1). Khi đó, tập phương án và hàm mục tiêu của bài toán được xác định như sau.

Tập phương án của bài toán: Rõ ràng, mỗi hành trình của người du lịch có dạng $X = (x_1, x_2, \dots, x_n, x_1)$. $X = (x_1, x_2, \dots, x_n)$ là hoán vị của $1, 2, \dots, n$ và $x_1 = 1$ ở vị trí đầu tiên và cuối cùng vì ta xem người du lịch xuất phát từ thành phố số 1 và sau đó người du lịch phải trở lại điểm xuất phát. Như vậy, ta chỉ có $(n-1)!$ hành trình thực sự. Vì vậy, tập phương án của bài toán là

$$D = \{ X = (x_1, x_2, \dots, x_n) : x_1 = 1; x_i \neq x_j : \forall i \neq j; i, j = 2, 3, \dots, n \}.$$

Hàm mục tiêu của bài toán: Chi phí của mỗi hành trình ứng với mỗi $X \in D$ là tổng chi phí đi từ thành phố thứ i đến thành phố thứ j . Do vậy hàm mục tiêu của bài toán là:

$$f(X) = \sum_{i=1}^{n-1} C[X[i]][X[i+1]] \dots C[X[n]][X[1]] \\ = C[X[1]][X[2]] + C[X[2]][X[3]] + \dots + C[X[n-1]][X[n]] + C[X[n]][X[1]]$$

Như vậy, bài toán người du lịch có thể phát biểu dưới dạng bài toán tối ưu tổ hợp sau:

$$\text{Tìm } \min \{ f(X) : X \in D \}$$

Bài toán cho thuê máy. Một ông chủ có một cái máy để cho thuê. Đầu tháng ông ta nhận được yêu cầu thuê máy của m khách hàng. Mỗi khách hàng i sẽ cho biết tập N_i các ngày trong tháng cần sử dụng máy ($i = 1, 2, \dots, m$). Ông chủ chỉ có quyền hoặc từ chối yêu cầu của khách hàng i , hoặc nếu nhận thì phải bố trí máy phục vụ khách hàng i đúng những ngày mà khách hàng này yêu cầu. Hỏi rằng ông chủ phải tiếp nhận các yêu cầu của khách thể nào để cho tổng số ngày sử dụng máy là lớn nhất.

Tập phương án của bài toán. Ký hiệu, $I = \{ 1, 2, \dots, m \}$ là tập chỉ số khách hàng, S là tập hợp các tập con của I . Khi đó, tập hợp tất cả các phương án cho thuê máy là

$$D = \{ J \subset S : N_k \cap N_p = \emptyset, \forall k \neq p \in J \}.$$

Hàm mục tiêu của bài toán. Với mỗi phương án $J \in D$, mỗi phương án cho thuê máy là tổng số ngày sử dụng máy theo phương án đó. Do vậy, hàm mục tiêu của bài toán là $f(J) = \sum_{j \in J} |N_j|$. Khi đó, bài toán có thể phát biểu dưới dạng bài toán tối ưu tổ hợp sau:

$$\max \{ f(j) : j \in D \}.$$

Bài toán phân công công việc. Có n công việc và n thợ. Biết c_{ij} là chi phí cần trả để thợ i hoàn thành công việc thứ j ($i, j = 1, 2, \dots, n$). Cần phải thuê thợ sao cho các công việc đều hoàn thành và mỗi thợ chỉ thực hiện một công việc, mỗi công việc chỉ do một thợ thực hiện. Hãy tìm cách thuê n nhân công sao cho tổng chi phí thuê thợ là nhỏ nhất.

Tập phương án của bài toán. Rõ ràng, mỗi phương án bố trí thợ thực hiện các công việc tương ứng với một hoán vị $\pi = (\pi(1), \pi(2), \dots, \pi(n))$. Do vậy, tập các phương án của bài toán là tập các hoán vị của $1, 2, \dots, n$.

Hàm mục tiêu của bài toán. Ứng với mỗi phương án, chi phí theo phương án là

$$f(\pi) = C_{\pi(1),1} + C_{\pi(2),2} + \dots + C_{\pi(n),n}.$$

Công việc	Thợ thực hiện
1	$\pi(1)$
2	$\pi(2)$
...	...